

Contents

Electrostatics Lab — Project Overview Document (POD)

Date Created: January 29, 2026 **Status:** Production-Ready (Core Objectives Complete) **Project Repository:** [electrostatics-lab-v3/electrostatics-lab/](#) **Technology Stack:** React 18 + Three.js + TypeScript + Vite **License:** MIT (Free for educational use)

Section 1: Executive Summary

Project Overview and Mission

Electrostatics Lab is an interactive 3D web-based visualization tool designed to make abstract electrostatic concepts tangible and explorable for physics students. The application renders electric fields, equipotential surfaces, and field lines in real-time, allowing students to develop spatial intuition about electromagnetic phenomena that textbook diagrams alone cannot convey.

Primary Mission: Transform physics education by enabling students to *see* and *interact with* electric fields, bridging the gap between mathematical formulas ($E = kQ/r^2$) and physical reality.

Current Status

[COMPLETE] Production-Ready & Feature-Complete for Core Educational Objectives

The electrostatics-lab project has achieved all primary development goals: - All 11 charge configurations fully implemented and validated - Physics calculations verified against analytical formulas - Multiple visualization modes operational and optimized - Teacher preset system in place for pedagogical guidance - Performance targets met (≥ 30 FPS on typical laptops) - Comprehensive documentation and validation complete

The application is ready for classroom use, student self-study, and institutional deployment.

Technology Stack Summary

Component	Technology	Purpose
Frontend Framework	React 18	Component architecture, state management

Component	Technology	Purpose
3D Graphics Engine	Three.js via React Three Fiber	WebGL rendering, GPU acceleration
Language	TypeScript	Type safety, IDE support, maintainability
Build & Dev	Vite	Fast development, optimized production builds
UI Controls	Leva	Interactive parameter panels
Physics	Custom implementations	Field calculations, RK4 integration, marching cubes

Codebase Size: ~3,300 LOC across models, components, and utilities

Key Innovation: Camera-Aligned Slice View

The **camera slice** feature solves a critical visualization problem: how to represent 3D equipotential “layers” clearly when semi-transparent 3D surfaces become muddy and overlapped.

Solution: A live 2D heat map cross-section that automatically rotates to stay perpendicular to the viewing direction. As you rotate the camera, the slice rotates with you, always presenting a clear, artifact-free view of the field structure.

This innovation exemplifies the pedagogical design principle of **multiple representations** — the same physical system can be viewed as: - Vector arrows (direction and magnitude) - Field lines (flux visualization) - 3D equipotential surfaces (nested structure) - 2D heat map slice (clean cross-section)

Section 2: Main Objectives

Core Development Objectives — All Complete [COMPLETE]

#	Objective	Success Criteria	Status
1	Implement physics-accurate field calculations	$E(x,y,z)$ and $V(x,y,z)$ match analytical formulas within 2%	[COMPLETE] Complete
2	Support 11 diverse charge configurations	Point charges, extended objects, plates all working	[COMPLETE] Complete
3	Render real-time 3D vector field	≥ 30 FPS with 8+ vector density on laptops	[COMPLETE] Complete
4	Integrate field line visualization	RK4 integration, proper seeding and termination	[COMPLETE] Complete
5	Generate 3D equipotential surfaces	Marching cubes algorithm, multiple levels, smooth rendering	[COMPLETE] Complete

#	Objective	Success Criteria	Status
6	Develop interactive 2D slice views	XY, XZ, YZ planes with heat maps and contours	[COMPLETE] Complete
7	Create camera-aligned slice innovation	Live camera-perpendicular heat map with contours	[COMPLETE] Complete
8	Validate against known physics	All 10 validation tests passed (see VALIDATION.md)	[COMPLETE] Complete
9	Optimize for classroom use	Teacher presets, clean UI, responsive controls	[COMPLETE] Complete
10	Document pedagogical value	Overview document, teacher guide, formula references	[COMPLETE] Complete

Section 3: Current Status

3.1 Overall Assessment

Status: Production-Ready and Validated

The electrostatics-lab application is **fully functional, physics-validated, and suitable for immediate deployment** in classroom and self-study environments. All core learning objectives have been achieved. The codebase is clean, well-structured, and maintainable for future enhancement.

3.2 What's Working

Comprehensive Charge Library (11 Configurations)

Point Charges (6 configurations): - Single positive and negative charges (Coulomb's law demonstration) - Electric dipole (field line curvature, near/far field behavior) - Two like charges (null point, field line repulsion) - Quadrupole (higher-order multipole moment, $1/r^3$ falloff) - Triangle arrangement (three-body superposition)

Extended Objects (3 configurations): - Finite rod / line charge (perpendicular bisector symmetry, off-axis behavior) - Charged ring (on-axis singularity, axial field formula) - Charged disk (near-field plate approximation, far-field point charge transition)

Plates (2 configurations): - Finite rectangular plate (edge effects, central uniformity) - Parallel plates / capacitor (uniform field demonstration, device principle)

Accurate Physics

- **Field calculations:** Direct summation/integration methods validated numerically
- **Singularity handling:** Softened distance $r \rightarrow (r^2 + \epsilon)^{1/2}$ with $\epsilon = 0.05$ prevents infinities while maintaining accuracy
- **Symmetry verification:** Dipole bisector, ring axis, disk transitions all match analytical expectations
- **Validation coverage:** 10 comprehensive tests (dipole symmetry, far-field scaling, on-axis formulas, perpendicularity, field line conservation)

Real-Time Interactivity

- **Live parameter adjustment:** Change charge magnitude, position, dimensions instantly
- **Smooth camera controls:** Rotate (left-drag), pan (right-drag), zoom (scroll)
- **Responsive visualization:** All rendering modes update in real-time (no delays or re-computation waits)
- **Performance:** 35-60 FPS across all configurations with default settings

Multiple Visualization Modes

- **Vector field glyphs:** 3D arrows with color-mapped magnitudes
- **Electric field lines:** Properly seeded from positive charges, follow RK4 integration
- **Equipotential surfaces:** 3D isosurfaces via marching cubes, semi-transparent rendering
- **2D slice views:** XY, XZ, YZ planes with heat maps and contour overlays
- **Camera slice:** Live camera-aligned heat map with dynamic rotation

GPU-Accelerated Rendering

- **Instanced geometry:** Vector glyphs use GPU instancing for efficient rendering of thousands of arrows
- **WebGL pipeline:** Three.js handles modern shader technology across browsers
- **Optimized transforms:** Lazy computation of expensive surfaces (field lines, isosurfaces)
- **Configurable resolution:** Users can trade accuracy for performance on slower machines

Teacher Presets System

- Pre-configured scenarios in `src/presets/teacher-presets.json`
- Pedagogical notes and “What to Notice” annotations
- One-click loading for classroom demonstrations
- Foundation for preset sharing and community building

Clean, Maintainable Codebase

- **Type-safe:** TypeScript throughout eliminates entire classes of runtime errors
- **Modular architecture:** Clear separation of concerns (models, components, utilities)
- **Extensible design:** New charge configurations can be added by implementing `FieldModel` interface
- **Well-documented:** Inline comments explain numerical methods and physics principles

High-Quality Documentation

- `README.md` with quick-start guide and feature overview
- `electrostatics-lab-overview.md` explaining educational objectives and pedagogical design
- `VALIDATION.md` demonstrating physics accuracy with test results
- Inline code comments for numerical methods and algorithms

3.3 Known Limitations

#	Limitation	Impact	Workaround
1	Mobile experience suboptimal	Touch controls limited, screen small, performance degraded	Use on desktop/laptop for best experience; touch optimization planned Phase 1
2	No export functionality	Can't save visualizations for presentations/homework	Phase 1 enhancement: PNG screenshots, GIF animations
3	No magnetic fields	Can't visualize Lorentz force, current-carrying wires	Phase 3 enhancement: B-field visualization, particle trajectories
4	No time-varying fields	Can't show AC sources, EM radiation, field oscillations	Phase 4 enhancement: Parametric time-dependent fields
5	No LMS integration	Can't embed in Canvas, Blackboard, or Moodle	Phase 4 enhancement: LTI standard implementation
6	No VR/AR capability	Can't use immersive headsets for exploration	Phase 4 enhancement: WebXR API support
7	No assessment features	Can't auto-grade student predictions or measurements	Phase 4 enhancement: Quiz integration, validation engine

All limitations are documented in the roadmap and planned for future phases based on educational priority and technical complexity.

3.4 Recent Progress

Development Timeline:

- **Initial implementation (2024):** Core physics engine, point charge models, basic visualization
- **Extended objects phase (2024):** Rod, ring, disk, plate geometries with validation
- **Visualization suite (2024):** Field lines, equipotentials, 2D slices, heat maps
- **Camera slice innovation (Early 2025):** Real-time camera-aligned slice with contours
- **Validation and documentation (Jan 2025):** Comprehensive physics validation, pedagogical design documentation
- **Production release (Jan 2026):** Feature-complete, validated, ready for classroom use

Key milestones: - All 11 charge configurations implemented and working - 10/10 physics validation tests passing - Performance targets achieved (≥ 30 FPS) - Teacher preset system operational - Complete documentation suite in place

Section 4: Future Enhancement Roadmap

The electrostatics-lab project is production-ready for its core educational mission. Future enhancements follow a **4-phase development plan** organized by priority, educational value, and technical complexity.

Phase 1: Enhanced Visualization & Export (2-4 weeks) | Priority: HIGH

Objective: Increase usability and reach by enabling easy content sharing and mobile accessibility.

Features

1.1 Image & Animation Export

- **PNG export:** `Canvas.toDataURL()` to save current view as screenshot
- **GIF export:** `CCapture.js` library for recording and encoding field line animations
- **MP4 export:** Optional, for higher quality presentations
- **Use cases:**
 - Students include visualizations in lab reports
 - Instructors add to lecture slides
 - Textbook authors generate accurate field diagrams
 - Social media/blog sharing of interesting configurations

Educational value: HIGH **Technical complexity:** LOW

1.2 Preset Import/Export

- **Export current configuration:** Save all parameters (charges, positions, visualization settings) as JSON
- **Import presets:** Load community presets or custom saved configurations
- **Shareable links:** Generate URL with encoded parameters for direct sharing
- **Use cases:**
 - Teachers create problem sets with specific configurations
 - Students share interesting “what-if” scenarios
 - Community library of pedagogically useful setups

Educational value: HIGH **Technical complexity:** LOW

1.3 Mobile Touch Optimization

- **Gesture support:** Pinch to zoom, two-finger rotation, tap and hold for long-press menu
- **Responsive layout:** Compact control panels for small screens
- **Touch event handlers:** Replace mouse-based orbit controls with touch-native implementation
- **Performance adaptive:** Auto-reduce vector density on mobile for smooth experience
- **Use cases:**
 - Students explore with tablets in study groups
 - Classroom iPad sharing during discussions
 - Self-paced learning on personal devices

Educational value: HIGH **Technical complexity:** MEDIUM

Technical Requirements

New files/modules: - `src/utils/ExportManager.ts` — Handle PNG/GIF export logic - `src/components/PresetPanel.tsx` — UI for import/export - `src/utils/PresetSerializer.ts` — Serialize/deserialize configuration state - Enhanced CSS media queries in `App.css` for mobile responsiveness

Dependencies: - `CCapture.js` (already optional in most Three.js projects) - Browser Canvas API (built-in) - No additional npm packages required

Browser compatibility: - Chrome 90+, Firefox 88+, Safari 14+, Edge 90+ - Mobile browsers: iOS Safari 14+, Android Chrome 90+

Success Metrics

- **Export usage:** >50% of monthly active users export at least once
 - **Mobile performance:** Maintain ≥ 20 FPS on iPad Air 2 and Android tablets
 - **Community engagement:** Preset sharing library established with 20+ community presets
 - **User feedback:** >4.0/5.0 rating on ease-of-use surveys
-

Phase 2: Interactive Extensions (4-8 weeks) | Priority: MEDIUM

Objective: Transition from passive observation to active experimentation by enabling students to create and measure custom configurations.

Features

2.1 Interactive Charge Placement Tool

- **Click-to-place:** Click in 3D space (with raycasting) to add new charges
- **Drag to move:** Move existing charges by clicking and dragging
- **Charge adjustment:** Right-click on charge to adjust magnitude
- **Delete:** Delete charges by selection or keyboard shortcut
- **Persistent custom configs:** Save and load student-created configurations
- **Use cases:**
 - Students build configurations matching given field line patterns
 - Predict field shape before revealing visualization
 - Explore “what if I move this charge?” without UI controls

Educational value: HIGH **Technical complexity:** MEDIUM

Implementation details: - Use Three.js Raycaster for 3D picking - State management for mutable charge array - Undo/redo stack for exploration comfort - Validation to prevent unphysical placements (optional constraint)

2.2 Measurement Tool (E-field & Potential Probe)

- **Click to probe:** Click at any point to read E and V values
- **Real-time display:**
 - Field magnitude $|E|$ in N/C
 - Components E_x , E_y , E_z

- Electric potential V in volts
- Position coordinates
- **Trajectory tracing:** Option to trace field line from probe point
- **Use cases:**
 - Verify that E is strongest near charges
 - Confirm perpendicularity ($dV/dn \sim E_n$)
 - Build quantitative understanding beyond qualitative visualization
 - Compare measurements across configurations

Educational value: HIGH **Technical complexity:** MEDIUM

Implementation details: - New component: `src/components/ProbeTool.tsx` - Extend Field-Model interface to provide precise E and V at arbitrary positions - 3D text labels for on-screen value display - History panel to compare measurements across multiple probes

2.3 Field Comparison Mode

- **Side-by-side rendering:** Two viewports, one configuration per side
- **Synchronized rotation:** Rotate one view automatically rotates the other
- **Toggle difference:** Option to show field difference ($E_1 - E_2$)
- **Use cases:**
 - Compare dipole orientation effects
 - Show how moving a charge affects overall field pattern
 - Demonstrate linear superposition by comparing (Config A + Config B) vs (Config A+B combined)
 - Explore symmetry breaking

Educational value: HIGH **Technical complexity:** MEDIUM

Implementation details: - Canvas split into two Three.js renderers - Shared lighting/camera logic for consistency - New state variable for comparison configuration

Technical Requirements

New files/modules: - `src/components/ChargePlacementTool.tsx` — Interactive placement UI - `src/components/ProbeTool.tsx` — Measurement interface - `src/components/ComparisonMode.tsx` — Side-by-side view - `src/models/CustomConfiguration.ts` — User-created charge arrangements - Enhanced `src/utils/FieldLineIntegrator.ts` for precision measurements

Dependencies: - Three.js Raycaster (built-in) - No new npm packages

Browser compatibility: - Same as Phase 1 (Chrome 90+, Firefox 88+, Safari 14+)

Success Metrics

- **Custom configs created:** Average user creates 3+ custom configurations per session
- **Measurement tool usage:** >40% of users perform at least one measurement
- **Learning gains:** Students who use interactive tools show 15% higher quiz scores on field concepts
- **Engagement:** Average session length increases 25% with Phase 2 features

Phase 3: Magnetic Fields & Dynamics (8-12 weeks) | Priority: MEDIUM

Objective: Extend to full electromagnetism by visualizing magnetic fields and charged particle dynamics in combined E+B fields.

Features

3.1 Magnetic Field Visualization

- **Biot-Savart law:** Compute B from moving charges and current distributions
- **Current-carrying wires:**
 - Straight wire: $B \sim 1/r$ (circulating field)
 - Circular loop: Dipole-like field pattern
 - Solenoid: Uniform interior field, dipole exterior
- **Visualization modes:**
 - Vector glyphs (like E-field)
 - Field lines (right-hand rule orientation)
 - Equipotential surfaces (isosurfaces of magnetic scalar potential in field-free regions)
- **Use cases:**
 - Visualize EM generator operation (moving charges $\rightarrow$ B-field)
 - Understand Ampere's law through symmetry
 - Show coupling between E and B in moving charge scenarios

Educational value: HIGH **Technical complexity:** HIGH

Implementation details: - New interface: `MagneticFieldModel` in `src/models/types.ts` - New directory: `src/models/magnetic/` - New models: - `StraightWire.ts` — Semi-infinite or finite current-carrying wire - `CircularLoop.ts` — Biot-Savart from closed loop - `Solenoid.ts` — Simplified solenoid approximation - Reuse visualization components (vector glyphs, field lines, isosurfaces) with B-field pipeline

3.2 Lorentz Force & Particle Trajectories

- **Force calculation:** $F = q(E + v \times B)$
- **ODE integration:** RK4 integration of equations of motion: $m \, dv/dt = q(E + v \times B)$
- **Trajectory visualization:**
 - Animated particle motion through E+B fields
 - Configurable initial conditions (position, velocity)
 - Trail rendering showing historical path
- **Quantitative output:** Particle energy, cyclotron frequency, radius of curvature
- **Use cases:**
 - Cyclotron motion in pure B-field (circle at $\omega_c = qB/m$)
 - Drift in crossed EB fields
 - Mass spectrometer operation
 - Magnetron and other particle devices

Educational value: HIGH **Technical complexity:** HIGH

Implementation details: - New component: `src/components/ParticleDynamics.tsx` - New utility: `src/utils/ParticleIntegrator.ts` (RK4 solver for charged particle) - Configurable

particle properties: mass, charge, initial position/velocity - Animation loop for trajectory playback - Energy conservation validation

3.3 Combined EM Visualization

- **Unified display:** Show E and B fields simultaneously with different color schemes
- **Coupling visualization:** How E and B interact to produce motion
- **Pedagogical modes:**
 - “Generator mode” — show B-field produced by moving charges in E-field
 - “Accelerator mode” — show force and acceleration in crossed fields
 - “Collision mode” — particle-particle interactions in E+B
- **Use cases:**
 - Understand relativistic EB transformation concepts
 - Visualize electromagnetic induction
 - Explore plasma physics basics

Educational value: MEDIUM **Technical complexity:** HIGH

Technical Requirements

New files/modules: - `src/models/types.ts` — Add `MagneticFieldModel` interface - `src/models/magnetic/StraightWire.ts` — Biot-Savart for straight wire - `src/models/magnetic/CircularLoop.ts` — Magnetic dipole from loop - `src/models/magnetic/Solenoid.ts` — Solenoid approximation - `src/components/MagneticFieldGlyphs.tsx` — Render B-field arrows - `src/components/ParticleDynamics.tsx` — Particle visualization - `src/utils/ParticleIntegrator.ts` — RK4 solver for motion - `src/App.tsx` — Add magnetic cases to `CaseType` union

Dependencies: - No new npm packages (all physics is custom-implemented) - Optional: `numeric.js` for matrix operations (if needed for relativistic transforms)

Browser compatibility: - Chrome 90+, Firefox 88+, Safari 14+ (same as current) - No new API requirements

Success Metrics

- **B-field accuracy:** Matches Biot-Savart law within 3% at test points
- **Trajectory validation:** Cyclotron frequency and radius match analytical formula
- **Visualization quality:** Field line density and smoothness equivalent to E-field
- **Performance:** ≥ 25 FPS with B-field visualization (acceptable reduction from E-field baseline)
- **Educational impact:** E&M physics courses adopt tool for EM induction and dynamics units

Phase 4: Platform Integration & Advanced Features (12-20 weeks) | Priority: LOW

Objective: Integrate into educational platforms and add cutting-edge immersive/adaptive features for maximum reach and effectiveness.

Features

4.1 LMS Integration (Canvas/Blackboard/Moodle)

- **LTI 1.3 standard implementation:**
 - OAuth 2.0 authentication
 - Tool Launch protocol
 - Assignment submission integration
 - Grade passback (if assessment is included)
- **Embedding:** Embed tool directly in course modules
- **Data integration:** Link to student identity for personalized progress tracking
- **Backend requirements:**
 - Node.js/Express server for LTI handshake
 - Database (PostgreSQL) for user and usage data
 - HTTPS only (LTI requirement)
- **Use cases:**
 - Students access tool from course shell (no separate app)
 - Assignments tied to specific configurations
 - Usage analytics for instructors
 - Integration with course gradebooks

Educational value: MEDIUM **Technical complexity:** HIGH

Implementation details: - New backend: `server/lti/` directory - LTI provider implementation (openid4java or custom Node library) - Token validation and session management - User provisioning and syncing - Frontend: Add session/user context to React app - Database schema for tracking assignments and attempts - Deployment: Containerize with Docker, deploy to institutional server

4.2 VR/AR Immersive Mode

- **WebXR API integration:** Use WebXR standard for VR/AR headsets
- **VR mode:**
 - Full 6-DOF head tracking
 - Hand controllers for interaction
 - Walk around field in 3D space
 - Grab and rotate charge objects
 - Teleportation for navigation
- **AR mode (mobile):**
 - Place field visualization in physical space
 - View from multiple angles using device motion
 - Overlay measurements on real world
- **Use cases:**
 - Immersive lab experience without expensive equipment
 - Group exploration in VR lab spaces
 - Remote collaboration via shared VR
 - Accessibility: students with mobility limitations can explore hands-free

Educational value: MEDIUM **Technical complexity:** HIGH

Implementation details: - WebXR API (Mozilla, Khronos standards) - Three.js WebXR examples as reference - Input handling for hand controllers - Spatial UI for parameter adjustment in VR -

Cross-browser support: Meta Quest (Android), HTC Vive, PlayStation VR, others

4.3 Time-Varying Fields

- **AC source modulation:** $E = E_0 \sin(t)$
- **Field oscillation:** Real-time animation of oscillating E-field
- **Wave propagation:** Traveling EM wave visualization
- **Parametric time-dependence:** Any field model can be multiplied by $f(t)$
- **Use cases:**
 - AC circuit visualization
 - Electromagnetic wave introduction
 - Radiation pattern from oscillating dipole
 - Phase relationships in EM fields

Educational value: HIGH **Technical complexity:** MEDIUM

Implementation details: - Extend `FieldModel` interface with optional time parameter: $E(\text{pos}, t)$ - Animation loop with time parameter - Visualization of wave fronts via phase coloring - Frequency controls in parameter panel

4.4 Assessment & Quiz Integration

- **Prediction challenges:** “What shape will this field have?” -> student predicts -> reveal visualization
- **Quantitative questions:** “At which point is $|E|$ maximum?” -> student places probe -> score feedback
- **Field line tracing:** “Sketch the field line starting here” -> compare to computed line
- **Concept validation:** Auto-check student understanding against physics rules
- **Grading integration:** Submit answers to gradebook via LTI
- **Use cases:**
 - Homework and quiz assignments
 - Pre-lecture concept checks
 - Post-lab validation
 - Formative assessment throughout course

Educational value: HIGH **Technical complexity:** HIGH

Implementation details: - New component: `src/components/AssessmentMode.tsx` - Question types: multiple choice, free response, interactive placement - Answer validation against physics (e.g., checking symmetry properties) - Backend: question repository and scoring engine - Integration with Phase 4.1 LMS for submission

Technical Requirements

New files/modules: - `server/` directory (Node.js backend): - `server/lti/LTIProvider.ts` — LTI handshake and validation - `server/auth/AuthManager.ts` — Session management - `server/db/` — Database models and queries - `server/api/` — REST endpoints for usage tracking - `src/contexts/UserContext.tsx` — User/session state in React - `src/components/AssessmentMode.tsx` — Quiz interface - Enhanced WebXR support (already built into Three.js)

Dependencies: - Backend: `express`, `passport` (OAuth), `pg` (PostgreSQL), `jsonwebtoken` - Frontend: `zustand` or `recoil` for complex state management (optional upgrade from current Leva) - WebXR: WebXR Emulator browser extension for testing (development only)

Infrastructure: - Backend server hosting (institutional or cloud) - PostgreSQL database - HTTPS certificate (required for LTI and WebXR) - Optional: Redis for session caching

Success Metrics

- **LMS deployment:** Tool integrated at 3+ institutions via LTI
 - **VR/AR validation:** Tested and working on 2+ headset platforms
 - **Time-varying fields:** Wave visualization validated against EM theory
 - **Assessment coverage:** 50+ question templates for various physics topics
 - **Usage analytics:** 80%+ of enrolled students complete at least one assignment
 - **Learning outcomes:** Students using assessment features score 20% higher on final exams
-

Section 5: Technical Architecture Impacts

This section details codebase changes and architectural decisions required for each phase.

Phase 1: Enhanced Visualization & Export

File Structure Changes:

```
src/
  utils/
    ExportManager.ts      [NEW]
    PresetSerializer.ts   [NEW]
  components/
    PresetPanel.tsx       [NEW]
  App.css                  [MODIFIED - mobile media queries]
```

Module Details:

src/utils/ExportManager.ts:

```
// Handle PNG and GIF export
export class ExportManager {
  exportPNG(canvas: HTMLCanvasElement, filename: string): void
  exportGIF(frameBuffer: Uint8Array[], filename: string): void
  recordFrames(callback: (frames: Uint8Array[]) => void): FrameRecorder
}
```

src/utils/PresetSerializer.ts:

```
// Serialize/deserialize application state
export interface SerializedPreset {
  caseType: CaseType
  parameters: Record<string, number>
  visualizationSettings: Record<string, boolean | number>
}
```

```

  metadata: { name: string; description: string; date: string }
}

```

CSS Responsive Changes: - Add media query breakpoints for tablets (768px) and phones (480px)
 - Responsive Leva panel: collapse to hamburger menu on mobile - Touch-friendly button sizes (min 44x44px) - Viewport meta tag handling for mobile browsers

Performance Considerations: - GIF encoding is CPU-intensive; offer quality/compression options - Export operates off main thread (Web Worker) to maintain 60 FPS - Stream large GIFs to disk incrementally (avoid memory spikes)

Phase 2: Interactive Extensions

File Structure Changes:

```

src/
models/
  CustomConfiguration.ts    [NEW]
components/
  ChargePlacementTool.tsx   [NEW]
  ProbeTool.tsx             [NEW]
  ComparisonMode.tsx        [NEW]
utils/
  FieldLineIntegrator.ts    [MODIFIED - add precision mode]
App.tsx                     [MODIFIED - add interaction modes]

```

Raycasting Implementation:

```

// In ChargePlacementTool.tsx
const raycaster = new THREE.Raycaster()
const mouse = new THREE.Vector2()

function onMouseDown(event: MouseEvent) {
  mouse.x = (event.clientX / window.innerWidth) * 2 - 1
  mouse.y = -(event.clientY / window.innerHeight) * 2 + 1

  raycaster.setFromCamera(mouse, camera)
  const intersects = raycaster.intersectObjects([groundPlane, ...chargeGeometries])

  if (intersects.length > 0) {
    selectedCharge = getChargeAtPoint(intersects[0].point)
  }
}

```

State Management for Mutable Charges: - Use `useState` hook in App component to manage custom charge array - Extend `FieldModel` to support dynamic charge repositioning - Implement undo/redo via state history stack

Measurement Tool Logic:

```
// In ProbeTool.tsx
function measureAt(position: THREE.Vector3) {
  const E = fieldModel.E(position)
  const V = fieldModel.V(position)
  return {
    position,
    magnitude: E.length(),
    components: { Ex: E.x, Ey: E.y, Ez: E.z },
    potential: V
  }
}
```

Phase 3: Magnetic Fields & Dynamics

File Structure Changes:

```
src/
models/
  types.ts           [MODIFIED - add MagneticFieldModel]
  magnetic/
    StraightWire.ts  [NEW]
    CircularLoop.ts  [NEW]
    Solenoid.ts      [NEW]
  combined/
    EMInteraction.ts [NEW]
components/
  MagneticFieldGlyphs.tsx [NEW]
  ParticleDynamics.tsx    [NEW]
  FieldLines.tsx          [MODIFIED - add B-field support]
utils/
  ParticleIntegrator.ts   [NEW]
App.tsx                   [MODIFIED - add magnetic cases]
```

New Interface in src/models/types.ts:

```
export interface MagneticFieldModel {
  name: string
  description: string

  B(pos: Vec3): Vec3 // Magnetic field

  // Optional: scalar potential (for field-free regions)
  Phi(pos: Vec3): number

  // Source geometry (current loops, wires)
  getSources(): MagneticSource[]
}
```



```
export interface MagneticSource {
  type: 'wire' | 'loop' | 'solenoid'
  position: Vec3
  current: number
  geometry: any // Wire path, loop radius, solenoid dimensions
}
```

Biot-Savart Implementation:

```
// In StraightWire.ts
function B_straightWire(pos: Vec3, wireStart: Vec3, wireEnd: Vec3, I: number): Vec3 {
  //  $B = (\mu I / 4\pi) * (dl \times r) / r^2 ds$ 
  // Discretize wire into segments and sum contributions

  const mu0 = 4 * Math.PI * 1e-7
  const segmentCount = 50
  const direction = wireEnd.clone().sub(wireStart).normalize()

  let B = new Vec3()
  for (let i = 0; i < segmentCount; i++) {
    const param = i / segmentCount
    const dl_pos = wireStart.clone().addScaledVector(direction, param * length)
    const dl = direction.clone().multiplyScalar(length / segmentCount)

    const r = pos.clone().sub(dl_pos)
    const r_mag = r.length()

    //  $dl \times r$ 
    const cross = dl.clone().cross(r.normalize())

    B.addScaledVector(cross, (mu0 * I) / (4 * Math.PI * r_mag * r_mag))
  }
  return B
}
```

Particle Integration:

```
// In ParticleIntegrator.ts using RK4
function integrateStep(
  particle: Particle,
  E: (pos: Vec3) => Vec3,
  B: (pos: Vec3) => Vec3,
  dt: number
) {
  //  $F = q(E + v \times B)$ 
  //  $a = F/m$ 

  const q_over_m = particle.charge / particle.mass

  const force = (pos: Vec3, vel: Vec3) => {
```

```

    const E_field = E(pos)
    const B_field = B(pos)
    const lorentz = E_field.clone().addScaledVector(B_field.clone().cross(vel), 1)
    return lorentz.multiplyScalar(q_over_m)
  }

  // Standard RK4: update position and velocity
  // ... (standard ODE integration code)
}

```

Phase 4: Platform Integration & Advanced Features

Backend File Structure:

```

server/
  src/
    lti/
      LTIProvider.ts
      TokenValidator.ts
      LaunchHandler.ts
    auth/
      SessionManager.ts
    db/
      models/
        User.ts
        Assignment.ts
        Submission.ts
      migrations/
    api/
      users.ts
      assignments.ts
      analytics.ts
    types/
      lti.ts
    app.ts
  config/
    database.ts
  package.json

```

Frontend WebXR Integration:

```

// In App.tsx or new VRMode component
async function enterVRMode() {
  if (!navigator.xr) {
    console.error('WebXR not supported')
    return
  }
}

```

```

const session = await navigator.xr.requestSession('immersive-vr', {
  requiredFeatures: ['local', 'dom-overlay'],
  domOverlayConfig: { root: document.body }
})

// Setup VR rendering with Three.js XRManager
renderer.xr.enabled = true
renderer.xr.setSession(session)
}

Assessment Framework:

// Question type system
export interface Question {
  id: string
  type: 'prediction' | 'measurement' | 'tracing' | 'multiple-choice'
  title: string
  description: string
  configuration: SerializedPreset
  expectedAnswer: Answer
  hint?: string
}

export interface Answer {
  type: string
  validate(studentAnswer: any, expected: Answer): { correct: boolean; score: number }
}

```

Section 6: Risk Assessment & Mitigation

#	Risk	Likelihood	Impact	Mitigation Strategy
1	Mobile performance degradation	HIGH	MEDIUM	Implement adaptive rendering: auto-reduce vector density on mobile; use performance monitoring API to track FPS; provide “performance mode” toggle
2	Magnetic field calculation performance	HIGH	HIGH	Implement B-field caching for static configurations; use approximations (dipole model) for far-field; test on target hardware early in Phase 3

#	Risk	Likelihood	Impact	Mitigation Strategy
3	LMS integration blocked by IT policies	MEDIUM	HIGH	Engage with institutional IT early; provide deployment options (self-hosted vs cloud); offer standalone version as fallback
4	Low VR/AR adoption	MEDIUM	MEDIUM	Start with AR (mobile WebXR) as easier entry point; build compelling demo experiences; partner with progressive institutions
5	Export browser compatibility issues	MEDIUM	LOW	Test export on all supported browsers; provide graceful degradation (export PNG if GIF fails); document browser requirements clearly
6	Unphysical configurations from interactive placement	LOW	MEDIUM	Implement optional validation rules; provide visual feedback for unphysical states; include “physics check” button with warnings
7	Scope creep on Phase 4 features	HIGH	MEDIUM	Define clear MVP for each phase; implement feature flags; prioritize LMS integration over VR/AR initially; track and adjust timeline based on complexity

Overall Risk Level: MEDIUM-LOW

The project has low technical risk (all core algorithms proven), medium integration risk (LMS/VR dependencies), and manageable scope risk (phase-based approach with clear checkpoints).

Section 7: Success Metrics

Success will be measured through quantitative metrics aligned with educational objectives.

Phase 1: Enhanced Visualization & Export

Metric	Target	Measurement Method
Export feature adoption	>50% monthly active users	Analytics tracking

Metric	Target	Measurement Method
Mobile session usage	>20% of total sessions	Browser user agent + analytics
Mobile FPS maintained	>=20 FPS on iPad Air 2	Real device testing, browser profiler
Preset sharing activity	20+ community presets created	Preset library database count
User satisfaction	>4.0/5.0 on usability	Post-use survey (10-question Likert scale)

Phase 2: Interactive Extensions

Metric	Target	Measurement Method
Custom configurations per session	Average 3+ per user	State tracking in app
Measurement tool engagement	>40% of users take probe measurements	Event logging
Learning outcome improvement	+15% on field concept quiz	Pre/post assessment comparison
Session length increase	+25% average session duration	Analytics timing
Interaction error rate	<5% invalid placements	Input validation logging

Phase 3: Magnetic Fields & Dynamics

Metric	Target	Measurement Method
B-field numerical accuracy	3% error vs Biot-Savart	Validation test suite
Cyclotron motion validation	Frequency matches $\omega_c = qB/m$ to 2%	Trajectory analysis
Field line quality parity	B-field lines as smooth as E-field lines	Visual comparison + metric
Trajectory FPS target	>=25 FPS with particle dynamics	Real-time performance profiling
Course adoption	E&M courses use Phase 3 features	Instructor survey / LMS logs

Phase 4: Platform Integration & Advanced Features

Metric	Target	Measurement Method
LMS deployment count	3+ institutional deployments	Institution signup tracking
VR/AR platform coverage	Working on 2+ headset platforms	Testing on Meta Quest, HTC Vive
Assessment question coverage	50+ validated question templates	Question repository count
Student engagement via LMS	80%+ of students complete assignments	LMS grade passback data
Learning outcome gain	+20% on EM concepts (vs non-users)	Institutional outcome assessment

Section 8: Timeline & Milestones

Overall Project Timeline

Current Status (Jan 2026): Core product complete, validation done, ready for enhancement phases.

Phase 1 (Enhanced Export) Expected Completion: April 2026 **Phase 2 (Interactive) Expected Completion:** August 2026 **Phase 3 (Magnetic Fields) Expected Completion:** December 2026 **Phase 4 (Integration) Expected Completion:** August 2027 (ongoing based on adoption)

Detailed Milestones

Phase	Milestone	Estimated Completion	Dependencies
Phase 1	Implement PNG export	Week 2 (Feb 2026)	None
	Implement GIF export with CCArchive.js	Week 3	Canvas API, CCArchive library
	Mobile touch controls & responsive CSS	Week 4	Touch event API
	Preset serialization/deserialization	Week 2	JSON parsing
	Testing & optimization on mobile devices	Week 5	iPad/Android test devices
	All Phase 1 features complete	Week 5 (Feb 2026)	All above milestones
Phase 1 Release			

Phase	Milestone	Estimated Completion	Dependencies
Phase 2	Implement raycasting & charge placement	Week 3 (Mar 2026)	Three.js Raycaster
	Build probe/measurement tool UI	Week 4	FieldModel interface finalized
	Implement side-by-side comparison mode	Week 5	Canvas rendering optimization
	Add undo/redo for custom configs	Week 6	State history management
	Comprehensive testing & user feedback	Week 7	Pilot user group
Phase 2 Release	All Phase 2 features complete	Week 7 (May 2026)	All above milestones
Phase 3	Design Magnetic-FieldModel interface	Week 2 (June 2026)	Review with physics advisor
	Implement Biot-Savart for straight wire	Week 3-4	Numerical validation
	Implement circular loop & solenoid models	Week 5	Numerical validation
	Build particle trajectory integrator	Week 6-7	Trajectory validation suite
	Visualize B-field glyphs & field lines	Week 7-8	Reuse component templates
	Performance optimization & testing	Week 9-10	Real hardware testing
Phase 3 Release	All Phase 3 features complete	Week 10 (Sept 2026)	All above milestones
Phase 4.1	Design LTI implementation & plan backend	Week 2 (Oct 2026)	Requirements gathering
	Build Express server & LTI provider	Week 3-5	OAuth2, JWT libraries

Phase	Milestone	Estimated Completion	Dependencies
	Implement database models (User, Assignment)	Week 6	PostgreSQL setup, migration tools
	Frontend session/user context integration	Week 7	React context or state lib
	Testing with Canvas/Moodle sandboxes	Week 8-9	LMS test instances
Phase 4.1 Release	LMS integration complete	Week 9 (Jan 2027)	All above milestones
Phase 4.2	WebXR API research & prototyping	Week 2 (Jan 2027)	Headset access (if available)
	Implement VR mode with hand controls	Week 3-5	WebXR, controller input handling
	Implement AR mode (mobile WebXR)	Week 6	Mobile WebXR polyfill
	Testing on available hardware	Week 7	Test device access
Phase 4.2 Release	VR/AR mode complete	Week 7 (Mar 2027)	All above milestones
Phase 4.3	Implement time-varying field interface	Week 2 (Mar 2027)	None (extension of existing)
	Build EM wave visualization	Week 3	Phase 3 complete
	Add oscillating field animations	Week 4	Animation framework
Phase 4.3 Release	Time-varying fields complete	Week 4 (Apr 2027)	All above milestones
Phase 4.4	Design assessment question framework	Week 2 (Apr 2027)	Educational research input
	Build prediction challenge UI	Week 3-4	Component development
	Implement measurement & validation scoring	Week 5	Answer validation engine

Phase	Milestone	Estimated Completion	Dependencies
	Create 50+ question templates	Week 6-8	Content creation (may be iterative)
Phase 4.4 Release	Assessment features complete	Week 8 (June 2027)	All above milestones

Critical Path Analysis: - Phase 1 is self-contained, lowest risk - Phase 2 depends on Phase 1 (preset system helps testing) - Phase 3 can proceed in parallel with Phase 2 (independent models) - Phase 4.1 (LMS) blocks Phase 4.3 (assessment integration) and Phase 4.4 effectively - Phase 4.2 (VR/AR) is independent, can run in parallel with Phase 4.1

Section 9: Open Questions & Design Decisions

Key questions requiring resolution before full roadmap commitment:

1. Magnetic Field Visualization: Integrated vs. Separate?

Question: Should B-field models be tightly integrated into the existing E-field UI, or presented as a separate mode?

Options: - **Integrated:** Add B-field toggle to existing visualizations, combined E+B rendering - Pro: Unified interface, immediate EB relationship visible - Con: More complex UI, potential visual clutter - **Separate:** Dedicated “Magnetic Fields” section with distinct models (wire, loop, solenoid) - Pro: Cleaner interface, natural pedagogical progression - Con: Less direct coupling, students might miss E-B relationship

Decision needed: Curriculum architect + UX designer review. Recommend **separate mode** for pedagogical clarity, but with linking features.

2. Export Format Priorities: PNG/GIF/MP4/3D?

Question: Which export formats provide highest educational value vs. development effort?

Format analysis: - **PNG:** Quick screenshots, low effort, high use frequency - **GIF:** Animated sequences, moderate effort, good for showing dynamics - **MP4:** High quality, high effort, less browser support - **3D (glTF/STL):** Exotic, high effort, limited student use cases

Decision needed: Phase 1 MVP should prioritize PNG + GIF. MP4/3D deferred to Phase 1.2 if demand demonstrated.

3. Mobile Performance Approach: Adaptive Rendering vs. Manual Tuning?

Question: How should mobile devices maintain performance?

Options: - **Adaptive:** Auto-detect FPS, reduce vector density automatically to maintain ≥ 20 FPS - Pro: Seamless user experience - Con: Complex to implement, unpredictable on edge cases -

Manual presets: “Performance Mode” button, users select quality level - Pro: Simple, predictable - Con: Requires user education, some devices over/underpowered for preset

Decision needed: Implement **adaptive system in Phase 1**, with manual override for power users.

4. VR Mode: WebXR vs. Native App?

Question: For Phase 4.2 VR support, should we use WebXR or build native apps?

Options: - **WebXR:** Web standard, works across headsets (Meta, HTC, Valve) - Pro: Single codebase, easy distribution - Con: Browser VR still emerging, performance concerns - **Native:** SteamVR SDK or Unreal Engine integration - Pro: Maximum performance - Con: Multiple codebases, complex distribution, licensing

Decision needed: **WebXR as primary (2027+)**, native as optional if performance insufficient. Test early with prototype.

5. Educational Effectiveness Validation: How Do We Measure Learning Gains?

Question: How will we validate that enhanced features actually improve student learning?

Measurement approaches: - **Pre/post assessment:** Concept inventories before and after using tool - **Comparison groups:** Control group (textbook only) vs. treatment (with tool) - **LMS integration analytics:** Track usage patterns vs. quiz performance - **Faculty feedback:** Instructor reports on student conceptual understanding

Decision needed: Plan formal study with institutional partners (university physics dept) for Phase 3 completion. Budget 2-3 months for IRB approval and data collection. Work with assessment specialist to design valid studies.

6. LMS Integration Level: Iframe/LTI/Deep Integration?

Question: How deeply should the tool integrate with LMS platforms?

Integration levels: - **Iframe embed:** Simple, minimal data sharing - Pro: Easiest to implement - Con: Limited LMS integration, no single sign-on - **LTI 1.3:** Standards-based, includes authentication and assignment integration - Pro: Professional, works with Canvas/Blackboard/Moodle - Con: More backend complexity, OAuth2 required - **Deep integration:** Custom Canvas/Blackboard plugins with grade passback, analytics - Pro: Maximum institutional integration - Con: Significant development effort, platform-specific code

Decision needed: LTI 1.3 as MVP for Phase 4.1, defer deep integration to Phase 4.2+ based on institutional demand.

Section 10: Resource Requirements

Phase 1: Enhanced Visualization & Export

	Category	Resource	Estimate	Notes
Development	Developer time		120-160 hours	~3-4 weeks at 40 hrs/week
	Testing & QA		40 hours	Mobile device testing, browser comp
Tools & Dependencies	CCapture.js		Free (open source)	GIF encoding library
	Testing devices		~\$500 (one-time)	iPad + Android tablet for mobile tes
	Deployment		Included	No new server needed
Documentation	Feature guides, user docs		20 hours	Export/preset tutorials
Total Cost			~\$5,000-6,000	Mostly developer labor

Phase 2: Interactive Extensions

Category	Resource	Estimate	Notes
Development	Developer time	200-280 hours	~5-7 weeks at 40 hrs/week
	UX design & prototyping	40 hours	Interactive feature flows
	User testing (pilot group)	30 hours	Observe students using tool
Tools & Dependencies	None additional	\$0	Raycasting built into Three.js
Testing & Validation	Physics validation suite	30 hours	Ensure measurements accurate
Documentation	Tutorials, feature guides	30 hours	Measurement tool training
Total Cost		\$10,000-12,000	Higher complexity justifies more testing

Phase 3: Magnetic Fields & Dynamics

Category	Resource	Estimate	Notes
Development	Developer time	300-420 hours	~8-10 weeks at 40 hrs/week
	Physics expertise consultation	40 hours	Review Biot-Savart implementation
	Numerical validation testing	80 hours	Comprehensive algorithm testing
Tools & Dependencies	None additional	\$0	Custom implementation
Performance	Hardware profiling	30 hours	GPU testing on target devices
Documentation	Physics background, formulas	40 hours	Biot-Savart and Lorentz force
Total Cost		\$15,000-18,000	Most complex phase, high testing burden

Phase 4: Platform Integration & Advanced Features

Phase 4.1: LMS Integration

Category	Resource	Estimate	Notes
Development	Backend developer (full-time)	480-640 hours	12-16 weeks
	Frontend developer	120 hours	User context, session mgmt
	DevOps/deployment	80 hours	Server setup, HTTPS, database
	LMS integration testing	100 hours	Canvas, Moodle, Blackboard sandboxes
Infrastructure	Server hosting	\$500-1000/month	Cloud VPS or institutional server
	PostgreSQL database	\$100-200/month	Database service (or self-hosted)
Tools & Dependencies	OAuth libraries, JWT	Free (open source)	passport.js, jsonwebtoken
Legal/Admin	Institutional agreements	Variable	Security review, data handling agreements

Category	Resource	Estimate	Notes
Total Cost		\$25,000-35,000	Significant backend investment

Phase 4.2: VR/AR Integration

Category	Resource	Estimate	Notes
Development	Frontend developer	160-240 hours	4-6 weeks
	VR UX designer	80 hours	Interaction design for headsets
	Testing on hardware	60 hours	Meta Quest, HTC Vive testing
Hardware for testing	VR headset (if not available)	\$500-2000 (one-time)	Meta Quest 3 or HTC Vive
Tools & Dependencies	WebXR polyfills	Free (open source)	webxr-polyfill
Documentation	VR user guide	20 hours	Controller mapping, comfort guidelines
Total Cost		\$8,000-12,000	Moderate effort, hardware-dependent

Phase 4.3: Time-Varying Fields

	Category	Resource	Estimate	Notes
Development	Developer time		100-140 hours	2.5-3.5 weeks
	Physics validation		30 hours	EM wave accuracy testing
Tools & Dependencies	None additional		\$0	Extension of existing framework
Documentation	EM wave physics guide		20 hours	Radiation, wave propagation
Total Cost			\$3,000-4,500	Low complexity, reuses existing

Phase 4.4: Assessment & Quiz Integration

	Category	Resource	Estimate	Notes
Development	Full-stack developer		240-320 hours	6-8 weeks
	Educational designer		100 hours	Question design, learning objectives
	Content creation		200-300 hours	50+ validated question templates
	Testing & validation		80 hours	Student pilot testing

	Category	Resource	Estimate	Notes
Tools & Dependencies	LMS integration (Phase 4.1)		Depends	LTI grade passback
Documentation	Assessment user guide		30 hours	Instructor and student guides
Total Cost			\$15,000-20,000	Content creation significant

Overall Resource Summary

Phase	Dev Cost	Infrastructure	Total	Timeline
Phase 1	\$5,000-6,000	Minimal	\$5,000-6,000	5 weeks
Phase 2	\$10,000-12,000	Minimal	\$10,000-12,000	7 weeks
Phase 3	\$15,000-18,000	Minimal	\$15,000-18,000	10 weeks
Phase 4.1	\$25,000-35,000	\$6,000-14,400/yr	\$31,000-49,400	16 weeks
Phase 4.2	\$8,000-12,000	Minimal	\$8,000-12,000	6 weeks
Phase 4.3	\$3,000-4,500	Minimal	\$3,000-4,500	3.5 weeks
Phase 4.4	\$15,000-20,000	Minimal	\$15,000-20,000	8 weeks
TOTAL (All Phases)	\$81,000-107,500	\$6,000-14,400/yr	\$87,000-121,900	~12 months

Team Composition Recommendation: - 1 senior full-stack developer (React/Node.js) — primary contributor across all phases - 1 mid-level frontend developer — Phase 1-2 focus, VR support Phase 4.2 - 1 DevOps/backend specialist — Phase 4.1 LMS integration - 1 physics/education advisor — consultation (part-time, 10-15 hrs/month) - 1 QA tester — mobile and cross-platform testing (shared across phases)

Cost can be reduced by: - Leveraging open-source contributions - Partnering with university (faculty involvement, student developer time) - Phased rollout with community feedback

Section 11: Implementation Readiness Checklist

Phase 1 Readiness (Ready to begin immediately)

- ☐ CCAapture.js library integrated into build
- ☐ Mobile testing devices procured or accessed
- ☐ Responsive CSS framework reviewed

- ☐ Export feature design finalized
- ☐ Preset serialization format documented

Phase 2 Readiness (Ready when Phase 1 complete)

- ☐ Three.js Raycaster examples reviewed
- ☐ UX mockups for interactive tools approved
- ☐ Physics validation test suite prepared
- ☐ Pilot user group recruited
- ☐ Measurement accuracy requirements specified

Phase 3 Readiness (Ready when Phase 2 complete)

- ☐ Biot-Savart algorithm references collected
- ☐ Magnetic field physics advisor assigned
- ☐ Numerical validation plan documented
- ☐ Particle integrator prototype written
- ☐ Hardware profiling tools available

Phase 4.1 Readiness (Ready when Phase 3 complete)

- ☐ LTI standard documentation reviewed
- ☐ Database schema designed
- ☐ OAuth2 flow diagrammed
- ☐ Target LMS platforms (Canvas, Moodle) sandboxes accessed
- ☐ Institutional partnerships established

Phase 4.2 Readiness (Can start in parallel)

- ☐ WebXR browser support verified
- ☐ VR/AR UX patterns researched
- ☐ VR headset access confirmed (if available)
- ☐ Hand controller mapping documented
- ☐ Comfort/motion sickness guidelines prepared

Phase 4.3 Readiness (Ready after Phase 3)

- ☐ Time-dependent field interface designed
- ☐ EM wave visualization algorithms prototyped
- ☐ Animation performance tested
- ☐ Wave propagation validation cases identified

Phase 4.4 Readiness (Ready after Phase 4.1)

- ☐ Assessment question taxonomy defined
- ☐ Learning objective mapping completed
- ☐ Question bank structure designed
- ☐ Instructor pilot group organized
- ☐ Validation rubric prepared

Section 12: Approval & Sign-Off

This Project Overview Document establishes the development roadmap for electrostatics-lab through 2027.

Current Status: [COMPLETE] Core product complete and validated (January 2026)

Next Steps: 1. **Immediate:** Begin Phase 1 (Enhanced Export) — 5-week sprint 2. **Quarterly reviews:** Assess progress, gather user feedback, adjust priorities 3. **Phase gate meetings:** Before starting each major phase, review success metrics and adjust scope if needed 4. **Community engagement:** Share roadmap with instructors and educators for feedback

Stakeholder Sign-Off:

Role	Name	Date	Comments
Project Lead		[Signature]	
Physics Advisor		[Signature]	
Education Partner		[Signature]	

References & Additional Resources

Physics References

- Griffiths, D. J. (2013). *Introduction to Electrodynamics* (4th ed.). Pearson.
- Serway, R. A., & Jewett, J. W. (2019). *Physics for Scientists and Engineers* (10th ed.). Cengage Learning.
- Jackson, J. D. (1998). *Classical Electrodynamics* (3rd ed.). Wiley.

Technical Documentation

- Three.js Documentation
- React Three Fiber Docs
- WebXR API Specification
- LTI 1.3 Standard

Related Projects

- PhET Interactive Simulations: <https://phet.colorado.edu/>
- Falstad Vector Field Simulator: <https://www.falstad.com/vector/>
- GeoGebra Applets: <https://www.geogebra.org/>

Document Version: 1.0 **Last Updated:** January 29, 2026 **Next Review:** April 30, 2026 (after Phase 1 completion) **Maintainer:** [Project Lead Name] **License:** CC-BY-4.0 (Share and adapt freely with attribution)

“Physics is best learned by doing — not just solving equations, but seeing how those equations shape the invisible world around us.” — Electrostatics Lab Mission